

Bike Fleet Operations Platform

חברת Bike Sharing מפעילה מערך להשכרת אופניים וכלי תחבורה שיתופיים במספר מוקדים שונים. המשתמשים יכולים לאתר כלי זמין, לשכור אותו ולהחזיר אותו בתחנות או בנקודות החזרה בהתאם למבנה המערכת.

בכל רגע מצב המערכת משתנה: כלי רכב נלקחים ומוחזרים, זמינות התחנות משתנה, ובחלק מהמיקומים עלול להיווצר מחסור בכלים או חוסר במקום להחזרה.

המערכת שלנו נועדה לאסוף את המידע התפעולי ממקור חיצוני, לעבד ולשמור אותו, לזהות שינויים במצב התחנות ולספק תמונת מצב עדכנית והיסטורית דרך REST API.

Business Goal

המערכת תאפשר:

- צפייה במצב הנוכחי של תחנות ומיקומים שונים.
- בדיקת זמינות של כלי רכב ומקומות החזרה.
- זיהוי תחנות שמתקרבות למצב ריק או מלא.
- מעקב אחר שינויים במצב התחנות לאורך הזמן.
- זיהוי נקודות הדורשות תשומת לב תפעולית.

Data

בפרויקט נעבוד בעיקר עם שני סוגי מידע:

Reference Data – מידע יחסית קבוע על התחנות, כגון מזהה, שם, מיקום וקיבולת.

Operational Data – מידע שמשתנה לאורך הזמן, כגון מספר האופניים ומספר מקומות ההחזרה הזמינים בתחנה.

במהלך הפרויקט נבנה בהדרגה Data Pipeline שמקבל את הנתונים, מעבד אותם, מזהה שינויים, שומר אותם ומאפשר לצרוך אותם דרך API.

Professional Concepts

לפני שמתחילים לעבוד על המערכת, חשוב להכיר מספר מושגים שילוו אותנו לאורך הפרויקט.

- **Reference Data**
מידע שמתאר entities יחסית יציבים ומשתנה בתדירות נמוכה.
 - **Operational Data**
מידע שמתאר את מצב המערכת בזמן מסוים ומשתנה לאורך הזמן.
 - **Data Ingestion**
תהליך הכנסת נתונים ממקור חיצוני אל המערכת שלנו.
 - **Data Validation**
בדיקה שהנתונים שהתקבלו תקינים וניתנים לעיבוד.
 - **Event Stream**
זרם של אירועים או עדכונים שעוברים בין רכיבי המערכת.
 - **Current State**
המצב האחרון הידוע של entity במערכת.
 - **Historical Data**
שמירת מצבים או שינויים לאורך הזמן לצורך מעקב וניתוח.
 - **Change Detection**
זיהוי האם המידע החדש שונה מהמצב האחרון הידוע.
 - **Deduplication**
מניעת עיבוד או שמירה מיותרים של מידע שכבר התקבל.
 - **Idempotency**
יכולת לעבד שוב את אותו input בלי ליצור תוצאה שגויה או כפולה.
-

External API Discovery

בפרויקט זה נעבוד עם מקור נתונים חיצוני המבוסס על תקן **GBFS – General Bikeshare Feed Specification**.

המטרה בשלב הזה היא להבין איך ניגשים ל-API חיצוני, איך מוצאים את ה-feeds הרלוונטיים ואיך בודקים את הנתונים שמתקבלים.

Step 1 – Read the Overview

פתחו את:

[/https://gbfs.org/get-started](https://gbfs.org/get-started)

התמקדו רק בהבנה של:

- מהו GBFS.
- מהו Feed.
- ההבדל בין `station_information` לבין `station_status`.

אין צורך ללמוד את כל התקן.

Step 2 – Open the Real Data

עבדו עם ה-feeds הבאים:

Station Information

https://gbfs.lyft.com/gbfs/2.3/bkn/en/station_information.json

Station Status

https://gbfs.lyft.com/gbfs/2.3/bkn/en/station_status.json

בשלב זה רק פתחו את הקישורים ובדקו את מבנה ה-JSON.

Step 3 – Send Requests from the Terminal

שלחו בקשת HTTP לכל אחד מה-feeds באמצעות `curl`.

- `curl <URL>`

ודאו שאתם מצליחים לקבל JSON תקין.

Step 4 – Identify the Important Fields

ב-`station_information` מצאו:

- `station_id`
- `name`
- `lat`
- `lon`
- `capacity`

ב-`station_status` מצאו:

- `station_id`
- `num_vehicles_available`
- `num_docks_available`

- is_renting
- is_returning
- last_reported

Goal

בסיום השלב אתם צריכים לדעת:

- איזה מידע מספק כל feed.
- איזה field מחבר בין שני ה-feeds.
- איזה מידע הוא Reference Data — מידע בסיסי יחסית ויציב.
- איזה מידע הוא Operational Data — מידע תפעולי שמשתנה בזמן אמת.

Data Storage Design

במערכת נשתמש ב־MySQL וב־MongoDB.

עליכם להחליט איזה סוג מידע יישמר בכל Database, בהתאם לאופי הנתונים ולדרך שבה המערכת תשתמש בהם.

בהחלטה שלכם התייחסו בין היתר ל:

- מבנה הנתונים והקשרים ביניהם.
- תדירות השינויים.
- מידע קבוע יחסית לעומת מידע שמצטבר לאורך הזמן.
- אופן הקריאה והעיבוד הצפוי של הנתונים.

בסיום השלב הגדירו בקצרה מה יישמר בכל Database ונמקו את הבחירה.

System Implementation Flow

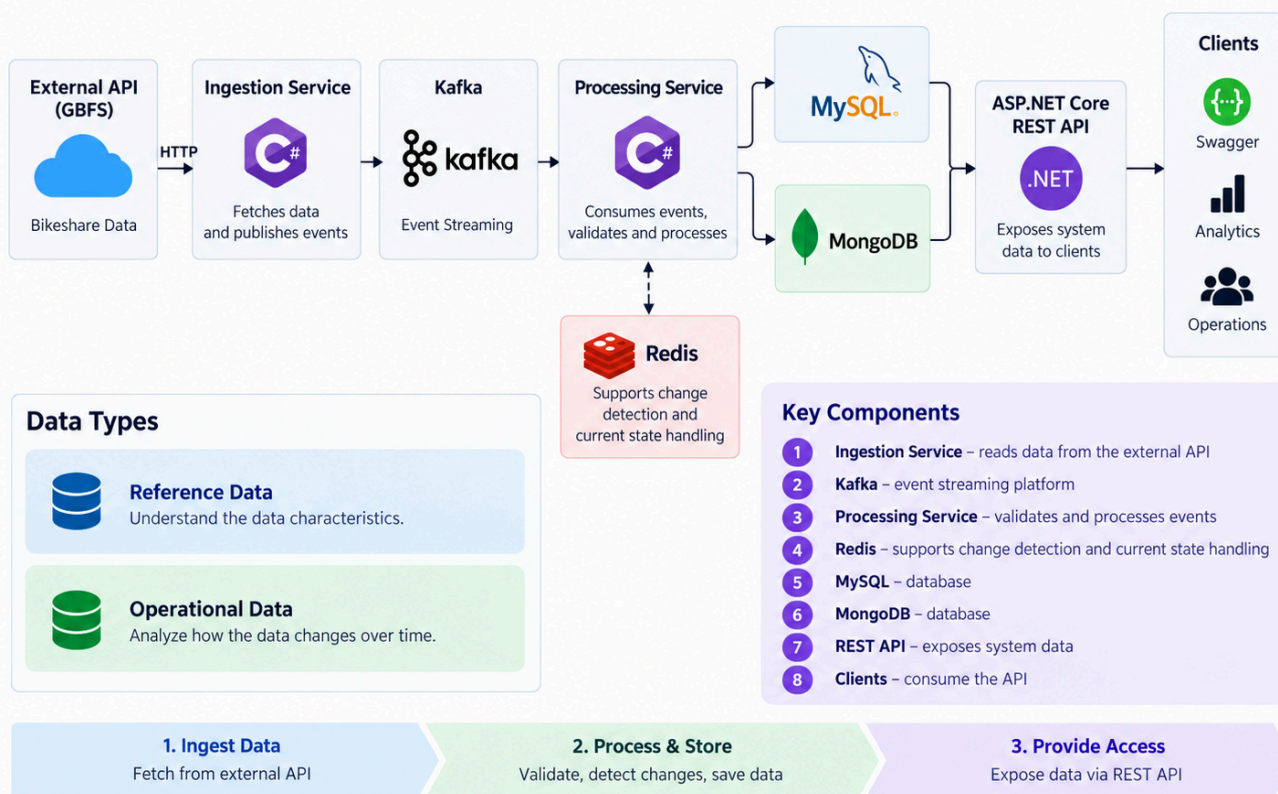
לאחר שהכרנו את מקור הנתונים והבנו את מבנה המידע שבו נשתמש, נעבור לבניית המערכת עצמה.

הדיאגרמה הבאה מציגה את ה־High-Level Architecture של המערכת ואת זרימת הנתונים בין הרכיבים המרכזיים.

System Architecture

High-Level Overview

The system ingests bikeshare data from an external API, processes it, stores relevant information and exposes it through a REST API.



מכאן נתקדם לפי ה-flow של המערכת, שלב אחר שלב.

בכל שלב נגדיר:

- מה האחריות של הרכיב.
- איזה מידע הוא מקבל ומעביר.
- אילו בדיקות וכללים עליו ליישם.
- כיצד הוא מתחבר לשאר ה-Data Pipeline.

המטרה היא לבנות כל רכיב בנפרד, ובסיום לחבר את כל הרכיבים למערכת אחת מלאה.

Ingestion Service

ה-Ingestion Service אחראי לקרוא את הנתונים מה-GBFS API ולהעביר אותם אל מערכת ה-Kafka.

בנו **C# Console Application** אחת, וכל התלויות בה ינוהלו באמצעות **Dependency Injection**.

Structure

צרו service נפרד לכל Feed:

- StationInformationService
- StationStatusService
- VehicleTypesService

כל service אחראי רק ל-Feed שלו: ביצוע הקריאה, המרת הנתונים למבנה #C מתאים, validation בסיסי ושליחת הנתונים ל-Kafka.

HTTP Requests

הקריאות ל-API יבוצעו באמצעות **HttpClient** המנוהל דרך **IHttpClientFactory** ו-**Dependency Injection**.

אין ליצור **HttpClient** חדש עבור כל בקשה.

Microsoft Reference:

<https://learn.microsoft.com/en-us/dotnet/core/extensions/httpclient-factory>

Polling Frequency

סוגי המידע במערכת אינם משתנים באותה תדירות, ולכן אין צורך לקרוא את כל ה-feeds באותו קצב.

- station_status – קריאה כל 60 שניות.
- station_information – קריאה בזמן עליית האפליקציה ולאחר מכן כל שעה.
- vehicle_types – קריאה בזמן עליית האפליקציה ולאחר מכן כל שעה.

station_status מייצג מידע תפעולי שמשתנה בתדירות גבוהה, בעוד ששאר ה-feeds מכילים מידע יציב יותר.

לצורך הרצת פעולות מחזוריות השתמשו ב-**PeriodicTimer**.

Microsoft Reference:

<https://learn.microsoft.com/en-us/dotnet/api/system.threading.periodictimer>

DTOs

לכל Feed צרו DTO שמתאים למבנה ה-JSON שמתקבל מה-API.

ה-DTO אינו יוצר מידע חדש ואינו משנה את הנתונים. מטרתו רק לייצג את ה-JSON כאובייקטים ב-C#, כדי שניתן יהיה לבצע validation ולעבוד עם הנתונים בצורה מסודרת.

לאחר ה-validation, הנתונים ממשיכים במערכת באותו מידע שהתקבל מהמקור.

Validation

לפני פרסום הנתונים ל-Kafka בצעו validation בסיסי בלבד:

- `station_id` חייב להיות קיים כאשר הוא נדרש לצורך זיהוי התחנה.
- `latitude` חייב להיות בטווח -90 עד 90.
- `longitude` חייב להיות בטווח -180 עד 180.
- ערכים כמו `capacity`, מספר כלי הרכב ומספר מקומות ההחזרה אינם יכולים להיות שליליים.
- אם בקשת HTTP נכשלה או שלא ניתן לבצע deserialization ל-JSON, אין לפרסם את הנתונים ל-Kafka.

שאר הנתונים מועברים כפי שהתקבלו מהמקור ואינם עוברים בשלב זה לוגיקה עסקית.

Output to Kafka

ה-Ingestion Service משמש כ-Kafka Producer.

לאחר שהנתונים עברו validation, כל Feed יפורסם ל-Topic נפרד:

- `bike.station-information`
- `bike.station-status`
- `bike.vehicle-types`

הפרדה זו מאפשרת לכל סוג מידע לעבור עיבוד עצמאי בהמשך ה-Data Pipeline.

ה-Ingestion Service אינו שומר מידע ב-Database ואינו מבצע Business Logic. האחריות שלו מסתיימת לאחר פרסום הנתונים התקינים ל-Kafka.

Processing Service

ה-**Processing Service** הוא Kafka Consumer שאחראי לצרוך את הנתונים משלושת ה-Topics ולעבד כל סוג מידע בהתאם לתפקידו במערכת.

צרו Handler נפרד לכל Topic:

- `StationInformationHandler`
- `StationStatusHandler`
- `VehicleTypesHandler`

Reference Data

הנתונים שמגיעים מ-`station-information` ומ-`vehicle-types` כבר עברו validation ב-Ingestion Service.

ה-Handlers שלהם אחראים לעדכן את ה-Database שנבחר עבור מידע זה:

- מידע חדש יתווסף.
- מידע שכבר קיים יעודכן במידת הצורך.

Station Status – Change Detection

`station_status` מתקבל בתדירות גבוהה, ולכן אותו מצב של תחנה יכול להגיע יותר מפעם אחת.

שמירת כל observation מחדש תיצור מידע כפול שאינו מוסיף ערך.

לכן השתמשו ב-Redis כ-**Current State Store** ושמרו עבור כל תחנה את המצב האחרון שטופל.

New Station Status

↓

Read Last State from Redis

↓

Compare

↓

No Change → Ignore

Changed → Continue

מנגנון זה נקרא **Change Detection**, והוא משמש לצורך **Deduplication** של observations שלא השתנו.

לפני שמירת שינוי ודאו שהתחנה שאליה מתייחס ה-status קיימת במערכת.

כאשר זוהה שינוי תקין:

1. שמרו את המצב החדש ב-Historical Storage.

2. עדכנו את ה-Current State ב-Redis.

כך נשמרת היסטוריה רק כאשר התרחש שינוי אמיתי במצב התחנה.

REST API Service

בנו **ASP.NET Core Web API** שחושף את הנתונים שכבר עברו עיבוד ונשמרו במערכת.

ה-API אינו פונה ישירות ל-GBFS ואינו צורך מ-Kafka.

Get Stations .1

GET /api/stations

מחזיר את רשימת התחנות.

ניתן לבצע filtering באמצעות הפרמטרים **האופציונליים** הבאים:

- status
- minAvailableBikes
- isRenting
- isReturning

ניתן לשלב מספר filters באותה בקשה.

אם לא נשלחו filters, יש להחזיר את כל התחנות.

Response

כל תחנה צריכה לכלול לפחות:

- id
- name
- latitude
- longitude
- capacity
- availableBikes
- availableDocks
- status
- isRenting
- isReturning

Get Station by Id .2

GET /api/stations/{id}

מחזיר תחנה אחת לפי המזהה שלה, כולל המידע הקבוע שלה והמצב הנוכחי.

Endpoint זה אינו כולל filtering.

אם התחנה אינה קיימת, יש להחזיר:

404 Not Found

Get Current Station Status .3

GET /api/stations/{id}/status

מחזיר את המצב הנוכחי של התחנה.

Response

- stationId
- availableBikes
- availableDocks
- isRenting
- isReturning
- status
- lastReported

אם התחנה אינה קיימת, יש להחזיר:

Not Found 404

4. Get Station History

GET /api/stations/{id}/history

מחזיר את היסטוריית השינויים של תחנה מסוימת.

ניתן להשתמש בפרמטרים האופציונליים הבאים:

- **from** – תחילת טווח הזמן.
- **to** – סוף טווח הזמן.
- **limit** – מספר הרשומות המקסימלי שיוחזר.

ניתן לשלב בין הפרמטרים.

Response

כל record צריך לכלול לפחות:

- timestamp
- availableBikes
- availableDocks
- status

Get System Dashboard .5

GET /api/dashboard

מחזיר תמונת מצב מסכמת של כלל המערכת.

Response

- totalStations •
- emptyStations •
- fullStations •
- lowAvailabilityStations •
- outOfServiceStations •
- lastUpdated •

החזירו **Response DTOs** המותאמים לכל endpoint ולא את מודלי ה-Database ישירות.

Bonus for Advanced Students – Location Services

עד עכשיו המערכת עובדת בעיקר עם `latitude` ו-`longitude`.

בשלב זה נרחיב אותה כך שתוכל לעבוד גם עם שמות של מקומות וכתובות באמצעות **Geocoding**.

לצורך הבונוס ניתן להשתמש ב-**Nominatim**, שירות Geocoding המבוסס על OpenStreetMap.

:Documentation

[/https://nominatim.org/release-docs/develop/api/Overview](https://nominatim.org/release-docs/develop/api/Overview)

Bonus 1 – Reverse Geocoding

הוסיפו אפשרות לקבל `latitude` ו-`longitude` ולהחזיר את שם המקום או הכתובת המתאימה.

GET /api/locations/reverse?lat={latitude}&lon={longitude}

לצורך זה השתמשו ב-Reverse Geocoding API:

[/https://nominatim.org/release-docs/develop/api/Reverse](https://nominatim.org/release-docs/develop/api/Reverse)

Bonus 2 – Search Stations by Coordinates

הוסיפו אפשרות לקבל נקודת ציון ולמצוא תחנות אופניים הנמצאות בקרבתה.

GET /api/stations/nearby?lat={latitude}&lon={longitude}&radiusKm={radius}

המערכת צריכה:

- לחשב את המרחק בין נקודת החיפוש לבין התחנות.
 - להחזיר רק תחנות שנמצאות בטווח המבוקש.
 - למיין את התחנות לפי מרחק.
-

Bonus 3 – Search by Place Name

הרחיבו את החיפוש כך שהמשתמש יוכל להזין שם מקום או כתובת במקום coordinates.

GET /api/stations/search?location={location}

המערכת תבצע:

Location Name

↓

Geocoding

↓

Latitude / Longitude

↓

Nearby Stations



Sort by Distance

לצורך המרת טקסט ל-coordinates השתמשו ב-Search API:

[/https://nominatim.org/release-docs/develop/api/Search](https://nominatim.org/release-docs/develop/api/Search)

התוצאה צריכה להחזיר את התחנות הקרובות ביותר ואת מצב הזמינות הנוכחי שלהן.

בעת שימוש בשרת הציבורי של Nominatim יש לקרוא את מדיניות השימוש ולכבד את מגבלת הבקשות והדרישה ל-User-Agent מתאים:

[/https://operations.osmfoundation.org/policies/nominatim](https://operations.osmfoundation.org/policies/nominatim)